

Veera's Student's Choice

2407011155

Name : M. Gowtham Raj

STD : ..... SEC : ..... ROLL NO : .....

S.No.	Date	Title	Page No.	Teacher's Sign /
1	23/7/26	Basic Networking Commands		
2	30/7/26	Types of Network cables		
3	30/7/26	CISCO Packet tracer tool study.		
4	10/8/26	Setting up a LAN		
5	10/8/26	Experiments on packet Capture tool.		
6	17/8/26	Hamming Code		
7	24/9/26	Sliding window protocol		
8	29/9/26	NMAP discover live host using Nmap Scans		
9	30/9/26	Implementation of subnetting in cisco packet tracer		
10a	30/9/26	Interconnecting with routers in cisco packet tracer		
10b	30/9/26	Design of router, cloud, DHCP		
11a	1/10/26	Simulate static routing in cisco packet tracer		
11b	1/10/26	Simulate RIP using cisco packet tracer.		
12	8/10/26	TCP, UDP		
13	9/10/26	ping program		
14	9/10/26	Packet Sniffing.		

Completed

17/7/26

Basic Networking Commands

Aim:

Study of various network Commands used in Linux and windows.

1) arp -a

Displays the address resolution protocol table, which contains the IP addresses and their corresponding MAC addresses.

OP:

Interface : 192 : 168 . 56 . 1 ---- 0xd

Internet address	physical address	Type
192.168.56.255	ff-ff-ff-ff-ff-ff	Static
224.0.0.22	01-00-5e-00-00-16	Static
224.0.0.251	01-00-5e-00-00-fb	Static

2) hostname:

Displays the hostname (Computer) of the local machine

OP:

LAPTOP-J8UHOT4B

3) ipconfig /all:

Displays detailed information about the network adapter including IP address, subnet mask

6 nslookup google.com

Queries the DNS server to address of a domain name.

OP:

Server: unknown
Address: 172.16.8.1
non authoritative answer:
Name: google.com
addresses: 2404:6800:4007:805::200e
142.250.67.46

7) pathping google.com

Combines the features of ping and tracert to trace the path to a destination and calculate packet loss at each hop.

OP:

Tracing route to google.com [142.250.67.46]
over a maximum of 30 hops:

0 LAPTOP-J80HOTJB [172.16.11.50]
1 172.16.8.1
2 137.97.163.129
3 * * *

8) ping google.com:

output

1: lo: <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default link/loopback
00:00:00:00:00:00:00:00:00:00:00:00
inet 127.0.0.118 scope host lo

11) ifconfig

Displays network interface details such as IP addresses and network statistics.

op:

eth0: flags=4163<UP, BROADCAST, RUNNING, MULTICAST> mtu 1500
inet 172.17.197.94 netmask 255.255.240.0 broadcast 172.17.205.255

12) mtr google.com

combines ping and traceroute to display the network path and performance statistics.

op:

LAPTOP-J8UHOTHB (172.17.197.94) → google.com

HOST	Loss%	Snt	Last
1 LAPTOP-J8UHOTHB.mshome.net	0.0%	12	0.7
2. 192.168.1.1	0.0%	46	4.7
3. 100.84.0.1	0.0%	163	5.7

13) tcpdump -i eth0:

Captures and displays network packets passing through a network interface for analysis

Op:

tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type,
capture size 262144 bytes
10:15:21.123456 IP 192.168.1.10.53134 >

14) Ping google.com:

sends ICMP echo request packets to verify network connectivity and measure response time.

Op:

PING google.com (142.251.43.206) 56(84) bytes.
64 bytes from iad30a-6b-in-f14.1e100.net (142.251.43.206): icmp_seq=1 ttl=117 time=5.76ms

Result:

Thus the various windows and linux networking commands were executed successfully.

Exp No: 2
Date: 30/07/26

Practical 2 - Types of Network Cables

Aim:

To study of different types of network cables

a) understand different types of network cable.

different type of cables used in networking are:

1) unshielded twisted pair (UTP) cable
2) Shielded twisted pair (STP) cable
3) Coaxial cable
4) Fibre optic cable.

Cable type: UTP

Category	Maximum Data Transmission
Category 3	10bps
Category 5	up to 100 Mbps
Category 5e	1 Gbps

Adv:

* Cheaper in Cost
* Easy to install as they have a smaller overall diameter

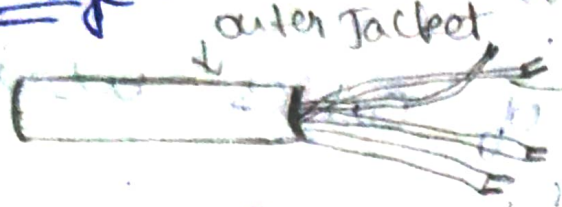
ai

+ more prone to (EMI) electromagnetic Interface and noise

Application

* 10 Base-T ethernet
* Fast ethernet

Image:

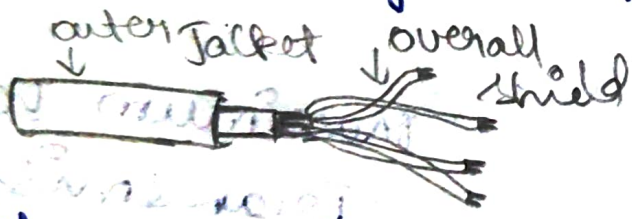


Cable type: STP

Category 6, 6a

Maximum Data transmission: 10Gbps

Application: Gigabit ethernet: 10G ethernet



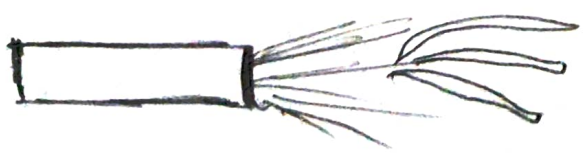
Cable type: SSTP

Category: Category 7

Maximum data Transmission: 10Gbps

Application: Gigabit ethernet, 10G ethernet

Image:



Advantages

* Shielded
* faster that UTP
* less susceptible to noise

* Disadvantage
  * Expensive
  * Greater Installation

Cable type: Coaxial Cable
Category: RG-6
RG-59
RG-11

Maximum Data transmission: 10-100 Mbps

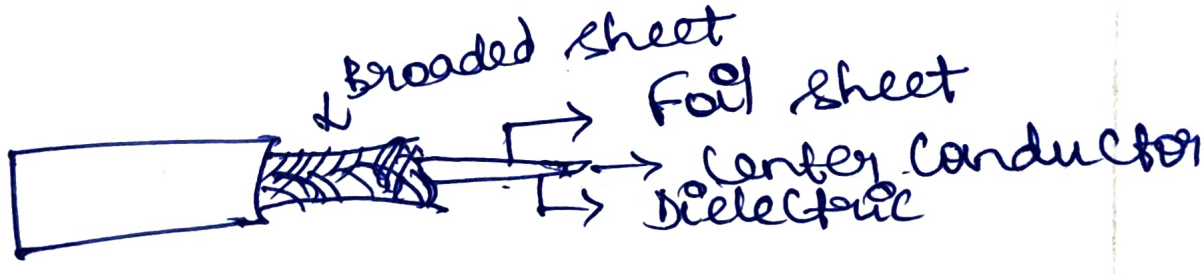
Advantages:
* High bandwidth
* Immune to interference



Dis
* Limited distance
* Cost

Application
Speed of signal is 500m Television network high Speed internet connection

Image



Cable type: fibre optics cable
Category: Single mode
Multi Mode

30/7/26 Practical - 3

Aim

To study the packet tracer tool installation and user interface overview

Introduction

A simulator, as the name suggests, simulates network devices and its environment.

* It allows you to model complex systems without the need for dedicated equipment.
* It helps you to practice your network configuration and troubleshooting skills via computer
* It is available for both the Linux and windows desktop environment

Analyse the behaviour of network devices using CISCO PACKET TRACER Simulator:

1. From the network component box, click and drag and drop the below components:
   a. 4 generic PC and one Hub
   b. 4 generic PC and one switch
2. Click on connections:
   a. Click on copper straight through cable.
   b. Select one of the PC and connect it to HUB.

using the cable.

4. Similarly connect 4PC to the switch using copper straight.

3. Click on the PCs connected to hub go to the desktop tab, click on IP configuration and enter an IP address and subnet mask.

4. observe the flow of PDU from source PC to destination PC by selecting the realtime mode of simulation.

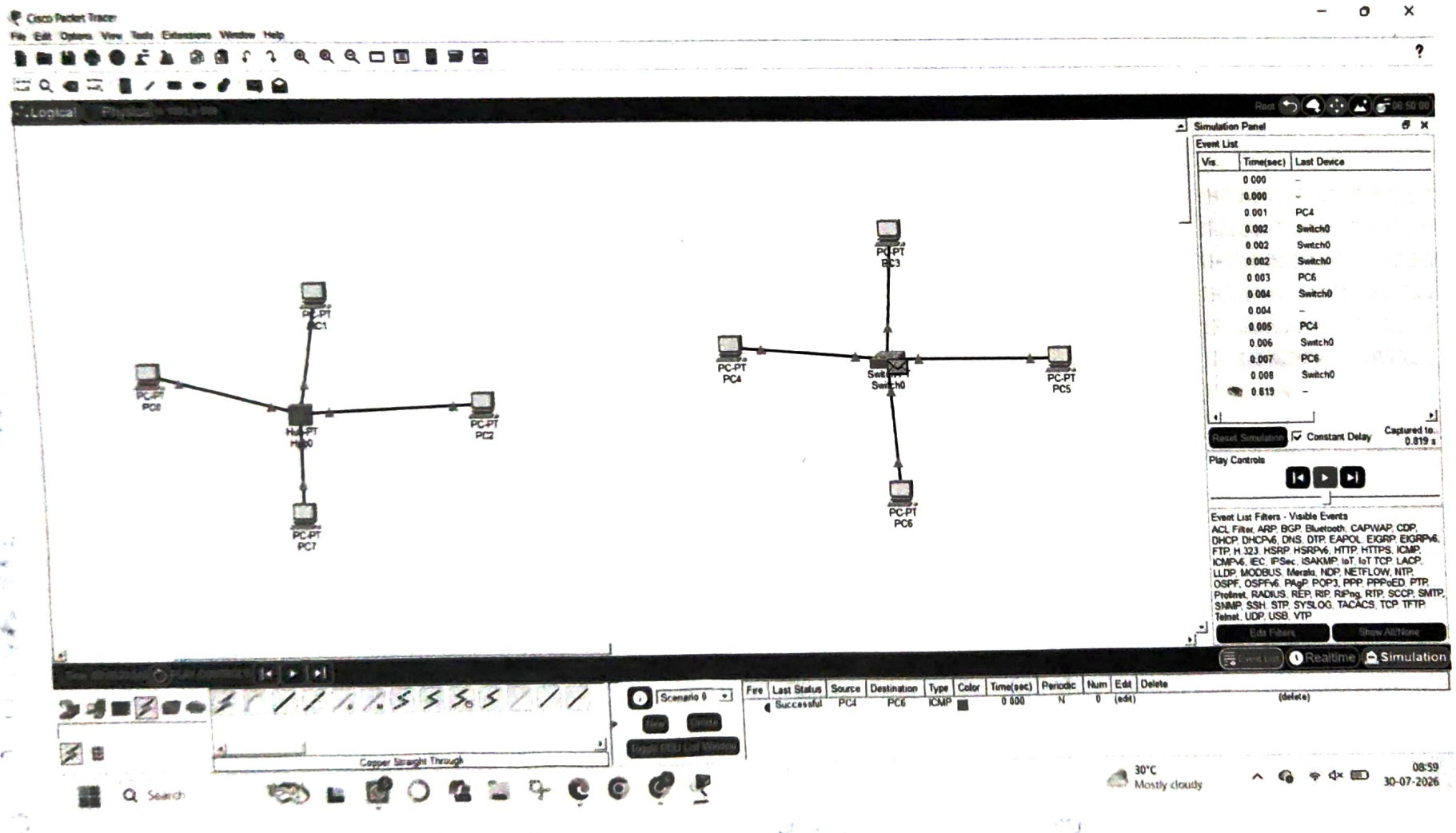
5. Repeat ~~step #~~ 3 to step#5 for the PCs connected to the switch

6. observe how HUB and switch are forwarding the PDU and write your observation and conclusion about the behaviors of switch and HUB.

OK

Result:

CISCO PACKET TRACER is learned and executed successfully.


Cisco Packet Tracer
File Edit Options View Tools Extensions Window Help
Logical
PC-PT PC1
PC-PT PC0
Hub-PT Hub0
PC-PT PC2
PC-PT PC7
PC-PT PC3
PC-PT PC4
Switch-PT Switch0
PC-PT PC5
PC-PT PC6
Simulation Panel
Event List
Vis. Time(sec) Last Device
0.000 –
0.000 –
0.001 PC4
0.002 Switch0
0.002 Switch0
0.002 Switch0
0.003 PC6
0.004 Switch0
0.004 –
0.005 PC4
0.006 Switch0
0.007 PC6
0.008 Switch0
0.819 –
Reset Simulation
Constant Delay
Captured to: 0.819 s
Play Controls
Event List Filters - Visible Events
ACL Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, DTP, EAPOL, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IEC, IPSec, ISAKMP, IoT, IoT TCP, LACP, LLDP, MODBUS, Meraki, NDP, NETFLOW, NTP, OSPF, OSPFv6, PAgP, POP3, PPP, PPPoED, PTP, Profinet, RADIUS, REP, RIP, RIPng, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP, USB, VTP
Edit Filters
Show All/None
Realtime
Simulation
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
Successful PC4 PC6 ICMP 0.000 N 0 (edit) (delete)
Copper Straight-Through
Search
30°C Mostly cloudy
08:59 30-07-2026

10/8/26

Practical 4

Setting up a LAN

Aim

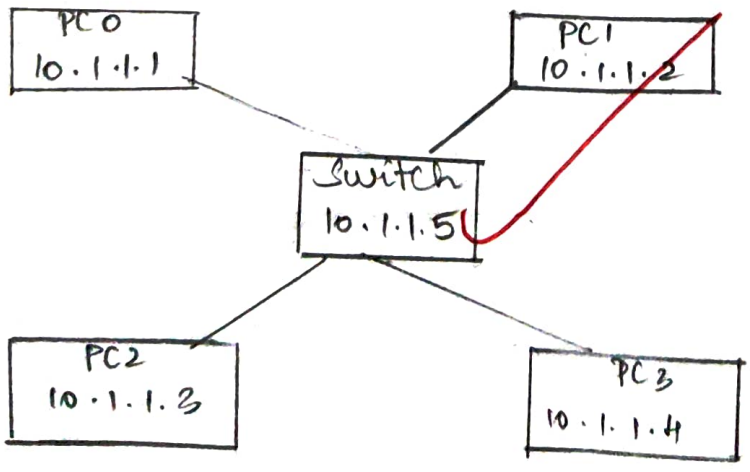
To design, set up and configure a LAN connecting four computers through a network switch, assign static IP to all devices.

Equipment Required

S.No	Equipment	Quantity	Specification
1.	PC	4	Ethernet enabled
2.	Network Switch	1	8/16/24 Port, unmanaged /managed
3.	Ethernet Cables	4	Straight through, RJ-45 Connectors.

Network Diagram

Star topology LAN



IP address Configuration table

Device	IP address	Subnet Mask	Connected To
PC 0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC 1	10.1.1.2	255.0.0.0	Fa0/2
PC 2	10.1.1.3	255.00.0	Fa0/3
PC 3	10.1.1.4	255.0.0.0	Fa0/4
Switch	10.1.1.5	255.0.0.0	VLAN1 Interface

Implementation in Cisco Packet Tracer

Since physical hardware was limited, the LAN was also implemented and verified using Cisco packet tracer simulation software

Step1: Place the devices

* open Cisco packet tracer
* Drag 4 PCs onto the workspace
* Drag 1 switch onto the workspace

Step2: Cable the devices

* Click Connections → Select Copper straight-through cable
* Click each PC then click switch and select an available fast

* Assign PT an IP address via PTs desktop → IP Configuration tab.
* open the server's services tab and enable the HTTP service.

Step7: observe Packet flow in simulation mode

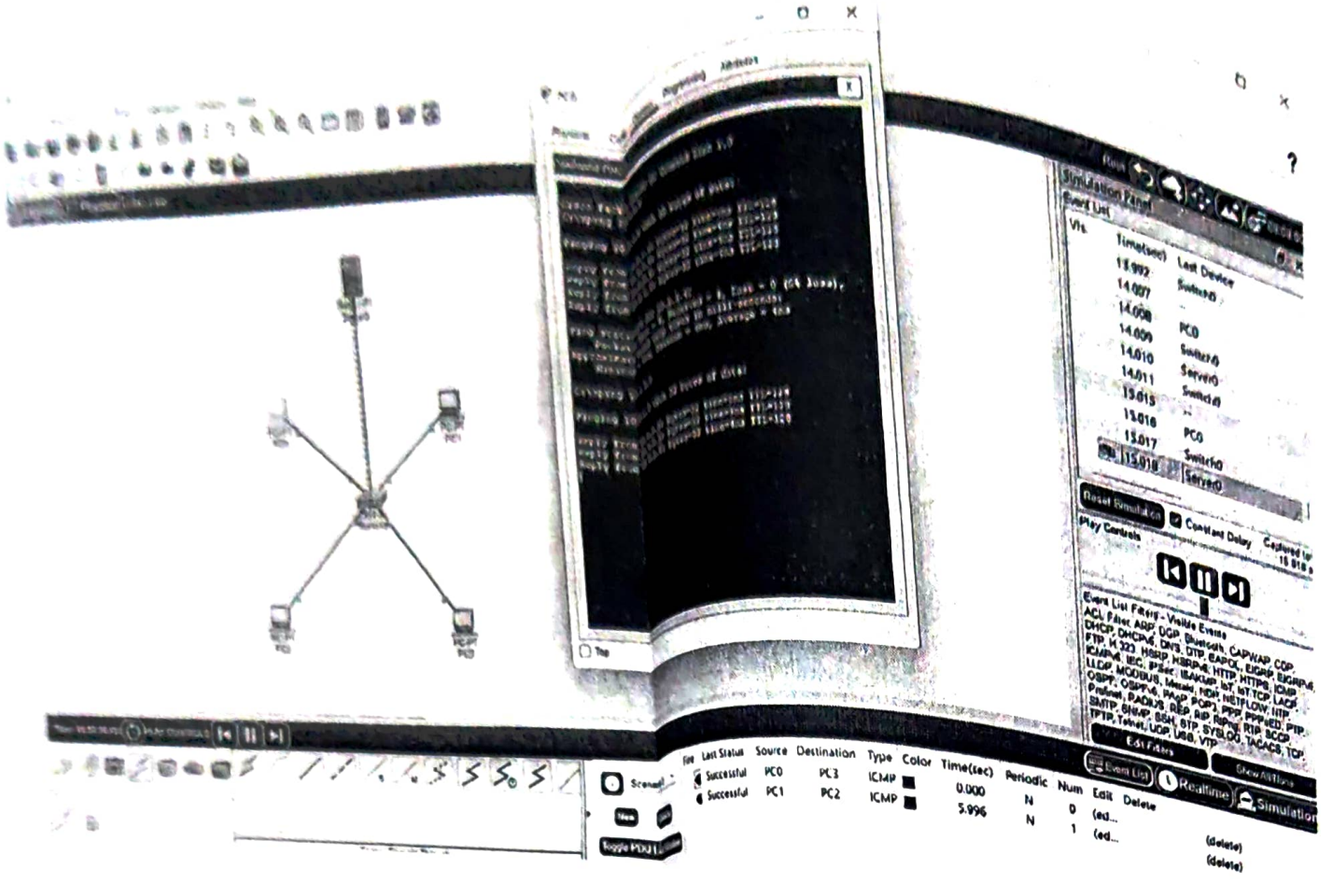
* switch to simulation mode
* Send a ping or web request between two devices and use the play/ capture controls to step through the packet as it travels hop-by-hop through the switch.

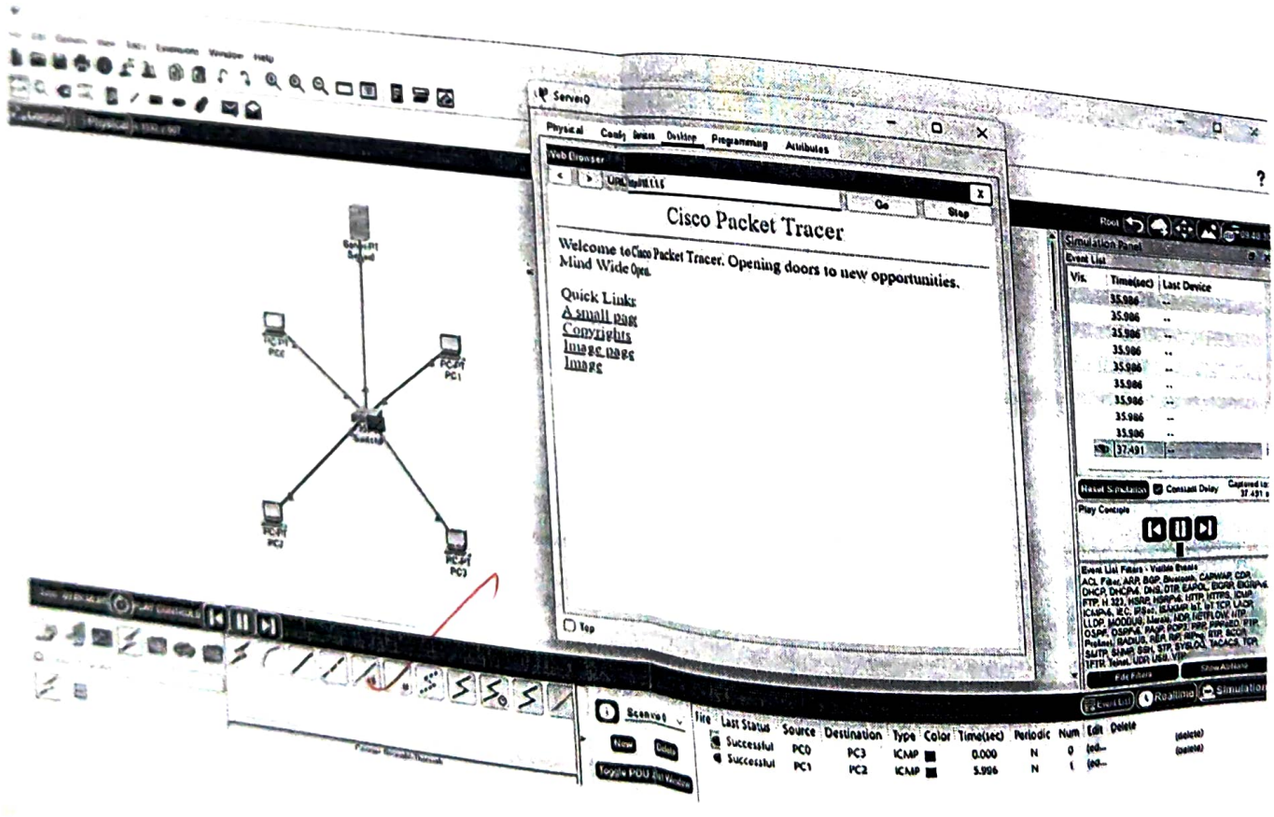
observation

All four PC's were successfully connected to the switch in a star topology and assigned unique static IP addresses with the same subnet.

Result:

Thus setting up LAN was executed successfully.




Cisco Packet Tracer
Welcome to Cisco Packet Tracer. Opening doors to new opportunities.
Mind Wide Open.
Quick Links:
A small page
Copyrights
Image page
Image

6/8/26

Exp: 5 Experiments on packet capture tool.

Aim: Experiments on packet capture tool: wireshark

wireshark:

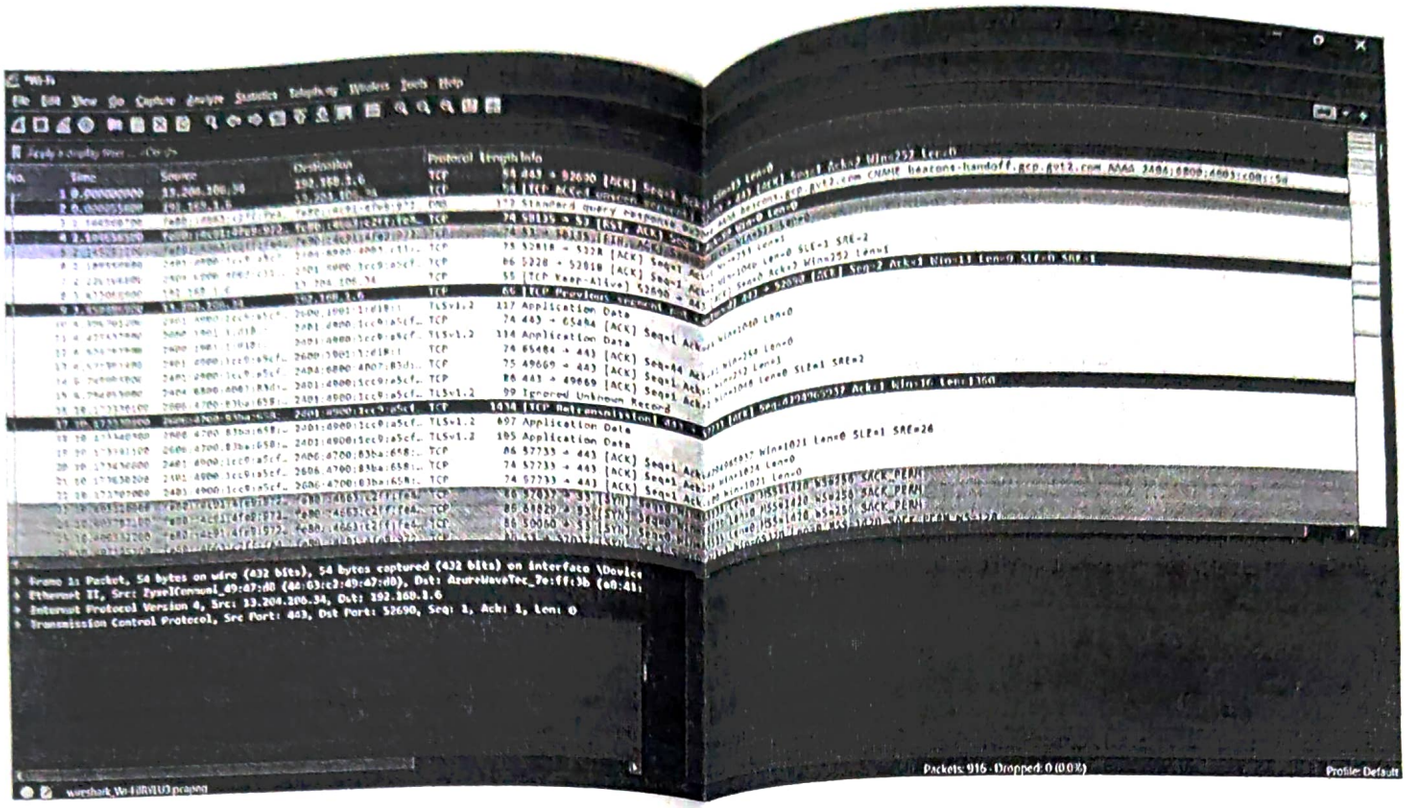
wireshark, a network analysis tool formally known as ethereal, capture packets in realtime and display them in human readable format.

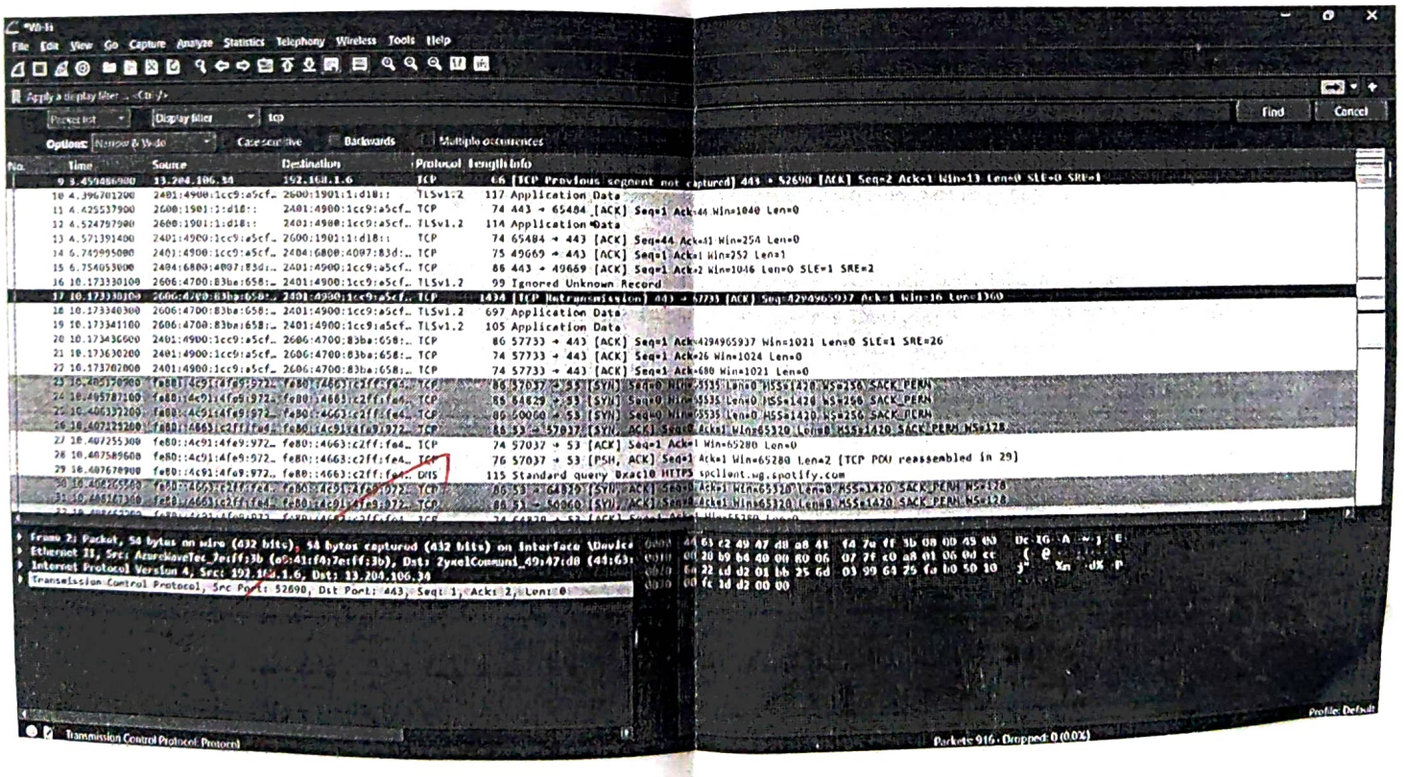
Procedure:

Confusing packets

After downloading & installing wireshark, launch it & double click the name of a network interface under capture to start capturing packets on that interfaces.

* wireshark captures each packet sent to or from your system.
* To check if promiscous mode is enabled, click > options and verify the enable promiscous mode on all interfaces" checkbox is activated at the bottom of this window. Create a filter to display only to TCP packet,


Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
Frame 1: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device
Ethernet II, Src: ZyxelCommuni_49:47:d0 (44:03:c2:49:47:d0), Dst: AzureWaveTec_9e:ff:3b (e0:41:
Internet Protocol Version 4, Src: 13.204.206.34, Dst: 192.168.1.6
Transmission Control Protocol, Src Port: 443, Dst Port: 52690, Seq: 1, Ack: 1, Len: 0
Packets: 916 · Dropped: 0 (0.0%)
Profile: Default


Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Display filter
tcp
Options: Narrow & Wide
Case sensitive
Backwards
Multiple occurrences
Find
Cancel
No. Time Source Destination Protocol Length Info
9 3.459486900 13.204.206.34 192.168.1.6 TCP 66 [TCP Previous segment not captured] 443 → 52690 [ACK] Seq=2 Ack=1 Win=13 Len=0 SLE=0 SRE=1
10 4.396701200 2401:4900:1cc9:a5cf... 2600:1901:1:d18::
11 4.425537900 2600:1901:1:d18:: 2401:4900:1cc9:a5cf... TCP 74 443 → 65484 [ACK] Seq=1 Ack=44 Win=1040 Len=0
12 4.524797900 2600:1901:1:d18:: 2401:4900:1cc9:a5cf... TLSv1.2 114 Application Data
13 4.571391400 2401:4900:1cc9:a5cf... 2600:1901:1:d18:: TCP 74 65484 → 443 [ACK] Seq=44 Ack=41 Win=254 Len=0
14 6.745995000 2401:4900:1cc9:a5cf... 2404:6800:4007:83d:... TCP 75 49669 → 443 [ACK] Seq=1 Ack=1 Win=252 Len=1
15 6.754053000 2404:6800:4007:83d:... 2401:4900:1cc9:a5cf... TCP 86 443 → 49669 [ACK] Seq=1 Ack=2 Win=1046 Len=0 SLE=1 SRE=2
16 10.173330100 2606:4700:83ba:658:... 2401:4900:1cc9:a5cf... TLSv1.2 99 Ignored Unknown Record
17 10.173330100 2606:4700:83ba:658:... 2401:4900:1cc9:a5cf... TCP 1434 [TCP Retransmission] 443 → 57733 [ACK] Seq=4294965937 Ack=1 Win=16 Len=1360
18 10.173340300 2606:4700:83ba:658:... 2401:4900:1cc9:a5cf... TLSv1.2 697 Application Data
19 10.173341100 2606:4700:83ba:658:... 2401:4900:1cc9:a5cf... TLSv1.2 105 Application Data
20 10.173436600 2401:4900:1cc9:a5cf... 2606:4700:83ba:658:... TCP 86 57733 → 443 [ACK] Seq=1 Ack=4294965937 Win=1021 Len=0 SLE=1 SRE=26
21 10.173630200 2401:4900:1cc9:a5cf... 2606:4700:83ba:658:... TCP 74 57733 → 443 [ACK] Seq=1 Ack=26 Win=1024 Len=0
22 10.173702800 2401:4900:1cc9:a5cf... 2606:4700:83ba:658:... TCP 74 57733 → 443 [ACK] Seq=1 Ack=680 Win=1021 Len=0
23 10.205170900 fe80::4c91:4fa9:972... fe80::4663:c2ff:fe4... TCP 86 57037 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1420 WS=256 SACK_PERM
24 10.405787100 fe80::4c91:4fa9:972... fe80::4663:c2ff:fe4... TCP 86 64829 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1420 WS=256 SACK_PERM
25 10.406317200 fe80::4c91:4fa9:972... fe80::4663:c2ff:fe4... TCP 86 50060 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1420 WS=256 SACK_PERM
26 10.407125200 fe80::4663:c2ff:fe4... fe80::4c91:4fa9:972... TCP 86 53 → 57037 [SYN, ACK] Seq=0 Ack=1 Win=65320 Len=0 MSS=1420 SACK_PERM WS=128
27 10.407255300 fe80::4c91:4fa9:972... fe80::4663:c2ff:fe4... TCP 74 57037 → 53 [ACK] Seq=1 Ack=1 Win=65280 Len=0
28 10.407589600 fe80::4c91:4fa9:972... fe80::4663:c2ff:fe4... TCP 76 57037 → 53 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=2 [TCP PDU reassembled in 29]
29 10.407670900 fe80::4c91:4fa9:972... fe80::4663:c2ff:fe4... DNS 115 Standard query 0xac10 HTTPS spclient.wg.spotify.com
30 10.408265500 fe80::4663:c2ff:fe4... fe80::4c91:4fa9:972... TCP 86 53 → 64829 [SYN, ACK] Seq=0 Ack=1 Win=65320 Len=0 MSS=1420 SACK_PERM WS=128
31 10.408617300 fe80::4663:c2ff:fe4... fe80::4c91:4fa9:972... TCP 86 53 → 50060 [SYN, ACK] Seq=0 Ack=1 Win=65320 Len=0 MSS=1420 SACK_PERM WS=128
Frame 2: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device
Ethernet II, Src: AzureWaveTec_9e:ff:3b (e0:41:f0:9e:ff:3b), Dst: ZyxelCommuni_49:47:d0 (44:03:
Internet Protocol Version 4, Src: 192.168.1.6, Dst: 13.204.206.34
Transmission Control Protocol, Src Port: 52690, Dst Port: 443, Seq: 1, Ack: 2, Len: 0
Transmission Control Protocol: Protocol
Packets: 916 · Dropped: 0 (0.0%)
Profile: Default

18bb Exp 06: Hamming Code

Aim:

To write a program to implement error detection and correction using hamming code.

Sender.py

```
def find_r(k):
    r=0
    while 2** r<k+r+1:
        r+=1
    return r

text = input("Enter text:")
binary =""
for ch in text:
    binary += format(ord(ch),"08b")
print("Binary:", binary)
k = len(binary)
r = find_r(k)
print("Data bits:", k)
print("Redundancy bits:", r)
n=k+r
code = [0]*(n+1)
j=0
for i in range(1,n+1):
    if i&(i-1)!=0:
        code[i] = int(binary[j])
        j+=1

p=1
```

```
while p<=n:
  parity=0
  for i in range(p,n+1):
    if i&p:
      parityn = code[i]
  code[p]= parity
  p= p*2
hamming_code = ""
for i in range(1,n+1):
    hamming_code += str(code[i])
print("Hamming code:", hamming_code)
with open("channel.txt","w") as f:
 f.write(str(K)+"\n")
 f.write(hamming_code)
print("Data saved in channel.txt")
```

Receiver.py

```
def binary_to_text(binary):
    text = ""
    for i in range(0,len(binary),8):
    byte = binary[i:i+8]
    if len(byte)==8:
    text += chr(int(byte,2))
  return text
with open("channel.txt","r") as f:
    K = int(f.readline())
    data = f.readline().strip()
```

```
print ("Received Hamming Code:", data)
n = len(data)
code = [0] + [int(x) for x in data]
Corrupted_binary = ""
for i in range(1, n+1):
    if i in range(1, n+1):
    if i & (i-1) != 0:
        Corrupted_binary += str(code[i])
Corrupted_binary = Corrupted_binary[:k]
Corrupted_text = binary_to_text(corrupted_
print ("Corrupted Text:", Corrupted_text)
                                binary)
error = 0
p = 1
while p <= n:
    parity = 0
    for i in range(p, n+1):
        if i & p:
            parity ^= code[i]
        if parity:
            error += p

        p = p*2
if error == 0:
    print ("No error detected.")
else:
    print ("Error detected at position:", error)
    code[error] = 1 - code[error]
    print ("error corrected.")
    corrected. library. = ""
for i in range (1, n+1):
```

```
corrected_binary += str(int(code[i]))
corrected_binary = corrected_binary[:k]
corrected_text = binary_to_text(corrected_binary)
print("Corrected Text:", corrected_text)
```

output:

```
Enter Text: HELLO
Enter data bits: 1011
Binary: 0100100001000010101001100010011000100111
Generated Hamming Code: 011001
Data Bits: 40
Redundancy Bits: 6
Hamming Code: 0101100010000010000101010011000110011001111
```

Receiver

```
Received code: 011001
No error detected.
Hamming Code: 0101100010000010000101010011000110011001111
corrupted text: HELNO
Error detected at: 37 position
Error corrected
Corrected Text: HELLO
```

Result:

Thus, error detection and correction is done using hamming code.

Exp 7 Sliding window Protocol

Aim: write a program to implement flow control at data link layer using sliding window protocol. Simulate the flow of frames from one node to another.

Code:

```
Import time
Import os

def write_file (filename, content):
    with open (filename, "w") as f:
        f.write(content)

def read_file (filename):
    if not os.path.exists (filename) or os.
                    path.getsize (filename) == 0:
        return None
    with open (filename, "r") as f:
        return f.read().strip()

def clear_file (filename):
    if os.path.exists (filename):
        open (filename, "w").close()

def main():
print ("=== Sender program ===")
window_size = int (input ("Enter window
                size (e.g., 3): "))
message = input ("Enter Text message (e.g.,
                HELLO): ")
```

```
total_frames = len(message)
base = 0
clear_file("Sender_Buffer.txt")
clear_file("Receiver_Buffer.txt")

while base < total_frames:
    buffer_content = ""
    current_window = []

    for i in range(base, min(base + window_
                    size, tot_frames)):
        buffer_content += f"{i}: {message[i]}
        current_window.append(i)      \n"

    print(f"\n[Sender] Sending window
                    frames: {current
                        _window}")
    write_file("Sender_Buffer.txt",
                    buffer_content.strip())

    print("[Sender] waiting 10 seconds")
    rx_reply = None
    for _ in range(5):
        rx_reply = read_file("receiver_
                        Buffer.txt")
        if rx_reply:
            break
        time.sleep(1)
    if rx_reply:
        print(f"[Sender] Response
                    received: {rx_reply}")
        try:
            status, ack_no = rx_reply.split(":")
```

```
        ack_no = int(ack_no)
        if status == "ACK":
            print(f"Success!")
            base = ack_no + 1
        elif status == "NACK":
            print(f"Error noticed!")
            base = ack_no
    except ValueError:
        print("[Sender] bad response")
    clear_file("Receiver_Buffer.txt")

else:
    print("[Sender] timeout!")

print("\n[Sender] Completed! All frames")
write_file("sender_Buffer.txt", "DONE")

if __name__ == "__main__":
    main()
```

Receiver

```
import time
import os
def write_file(filename, content):
    with open(filename, "w") as f:
        f.write(content)

def read_file(filename):
    if not os.path.exists(filename) or os.
                path.getsize(filename)
                == 0:
```

```
for line in lines:
    if not line: continue
    try:
        frame_no, character = line.split(":")
        frame_no = int(frame_no)
        if frame_no == expected_frame and
                                not error_found:

            print(f"Received frame {frame_no}
                    '{character}' -> [OK]")

            last_good_frame = expected_frame
            expected_frame += 1
            received_message.append(character)

        else:
            print(f"Received frame {frame_no}:
                    '{character}' -> [Error] Expected:
                                {expected_frame}")

            error_found = True
    except ValueError:
        error_found = True

    if error_found:
        print(f"--> Replying with NACK:
                                {expected_frame}")

        write_file("Receiver_Buffer.txt",
                    f"NACK:{expected_
                                frame}")

    else:
        if last_good_frame >= 0:
            print(f"--> Replying with ACK:{last_
                                good_frame}")
```

```
        write_file ("Receiver_Buffer.txt",
                    f"Ack: {last_good_
                            frame}")
        print(f"Current Received string: {"".
                    join(received_message)}")
        clear_file("Sender_Buffer.txt")

if __name__ == "__main__":
    main()
```

output:

Receiver program

Receiver waiting for incoming frame

Sender program

Enter window size = 3:

Enter text message = "HELLO"

Frames: [0, 1, 2]

Processing data from incoming frames

Received frame 0: 'H' → [OK]

Received frame 1: 'E' → [OK]

Received frame 2: 'L' → [OK]

---> Replying with ACK: 2

Current Received string: 'HEL'

Result:

Thus program for sliding window protocol has been executed successfully

Exp: 08 Nmap using Try hackMe
Date: 3/10/26

Aim

To discover live host on a network using Nmap host discovery scans - ARP scan, ICMP scan and TCP/UDP ping scan and to understand which protocol operates on which layer.

ARP host discovery

$ sudo nmap -PR -sn 10.200.6.0/24

Op

Starting Nmap 7.80 (https://nmap.org)
Nmap scan report for 10.200.6.1
Host is up
Nmap done: 256 IP address (1 host up)
scanned in 10.51 seconds

ICMP host discovery

ICMP Echo (-PE)

$ sudo nmap -PE -sn 10.200.6.0/24
Nmap scan report for 10.200.6.1
Host is up (0.069P latency)
Nmap scan report for 10.200.6.50
Host is up (0.070 latency)
Nmap done: 256IP address (3 hosts up)
scanned in 3.38 seconds.

ICMP Timestamp (-PP)

```
$nmap -PP -Sn 10.200.6.0/24
Nmap Scan report for 10.200.6.50
Host is up (0.069s latency)
Nmap Scan report for 10.200.6.250
Host is up (0.069s latency)
Nmap done: 256 IP address (3 hosts up) scanned in 3.44s
```

ICMP Address mask (-PM)

```
$ nmap -PM -Sn 10.200.6.0/24
Nmap done: 256 IP address (0 hosts up) scanned in 52.4s
```

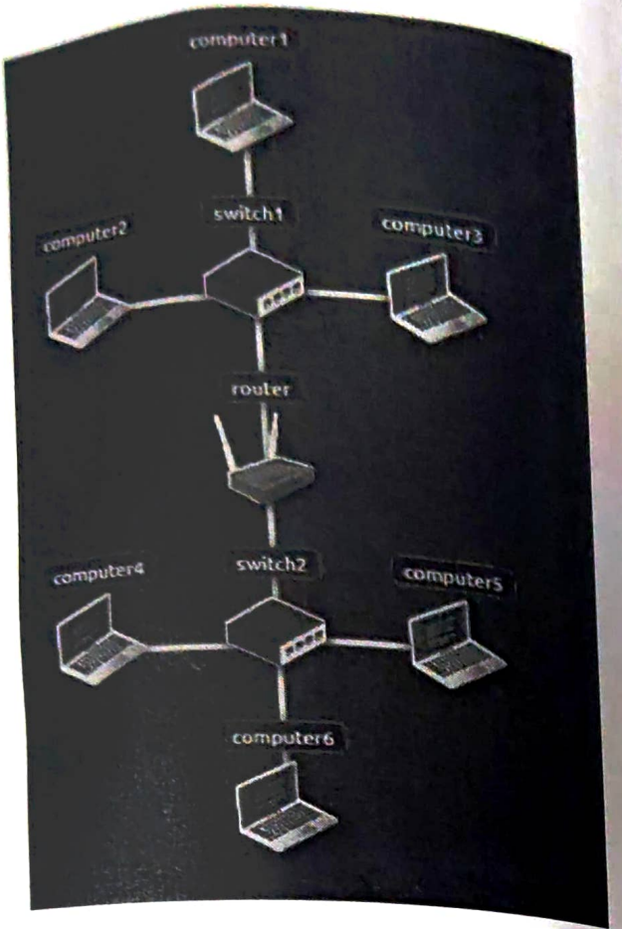
TCP UDP ping discovery

i) TCP SYN ping (-PS)

```
$nmap -PS -Sn 10.200.6.0/24
Nmap Scan report for 10.200.6.50
Host is up (0.07s latency)
Nmap Scan report for 10.200.6.250
```

(ii) UDP ping

```
$ sudo nmap -PU -Sn 10.200.6.0/24
Nmap Scan report for 10.200.6.50
Host is up (0.07s latency)
Nmap Scan report for 10.200.6.250
Host is up (0.069s latency)
Nmap done: 256 IP address (2 host up) scanned in 35.97s
```



Answer the questions below

How many hosts are found alive after scanning the CONNECTION_IP/24?

✓ Correct Answer

What is the option required to tell Nmap to use ICMP Timestamp to discover live hosts

✓ Correct Answer

What is the option required to tell Nmap to use ICMP Address Mask to discover live hosts

✓ Correct Answer

What is the option required to tell Nmap to use ICMP Echo to discover live hosts

✓ Correct Answer

Which TCP ping scan does not require a privileged account?

✓ Correct Answer

Which TCP ping scan requires a privileged account?

✓ Correct Answer

What option do you need to add to Nmap to run a TCP SYN ping scan on the ta

✓ Correct Answer

Exp no : 4
3/10/26

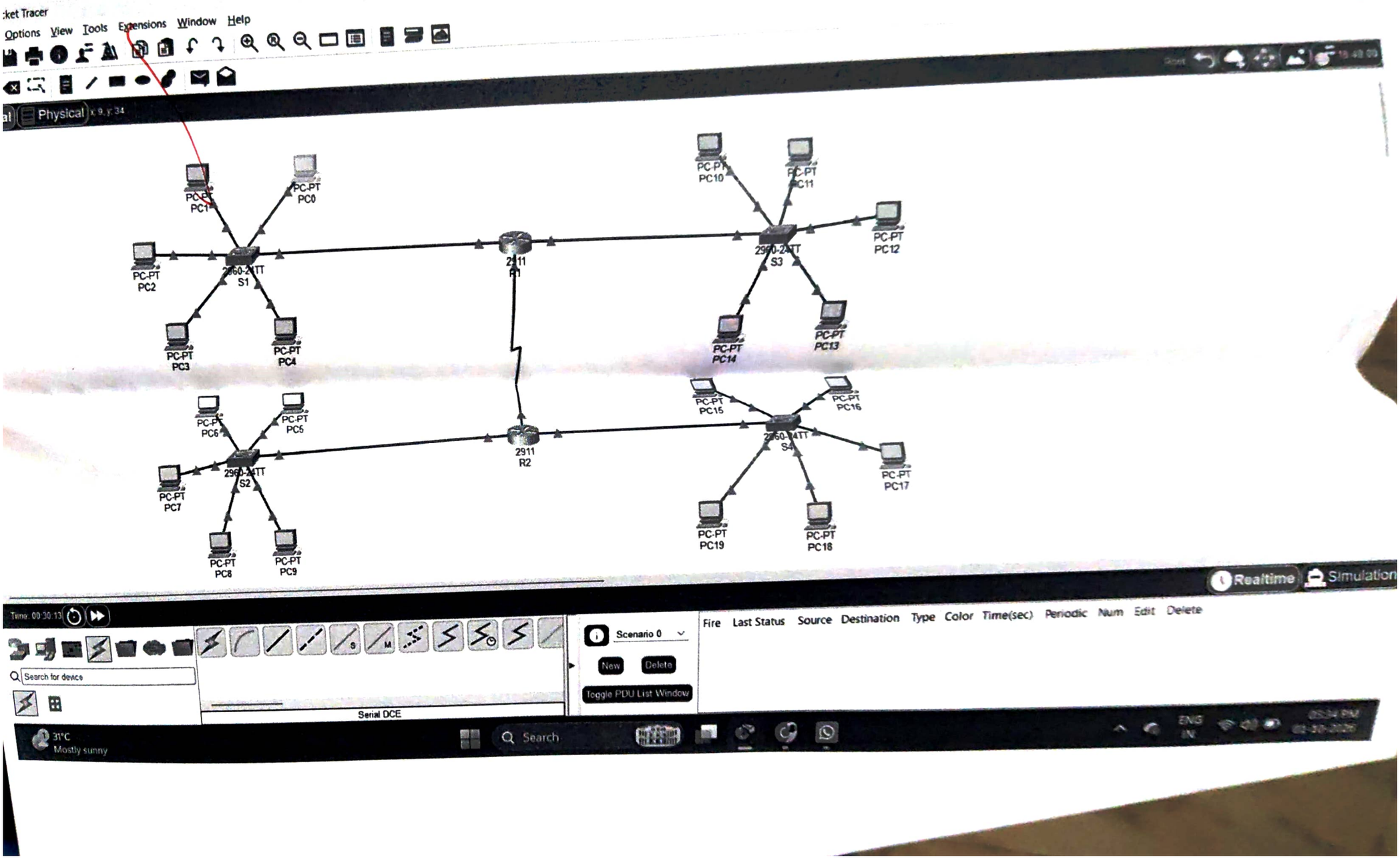
Implementation of subnetting in cisco packet Tracer simulator

Aim

To implement the subnetting in cisco packet tracer simulator

Procedure:

* open cisco packet tracer and create a new network topology
* Add the required routers, switches and PCs to workspace
* Connect the device using appropriate copper cables.
* use the network 192.168.1.0/24 and divide into 128 network
* use enable and config terminal in the router CLI
* Configure the required switch port in access mode
* Assign each pc its IP address, subnet mask and default gateway
* Repeat the configuration all required subnet
* use the ping command to test connectivity between PCs and the routers.


Options View Tools Extensions Window Help
Physical
PC-PT PC0
PC-PT PC1
PC-PT PC2
2960-24TT S1
2911 R1
PC-PT PC3
PC-PT PC4
PC-PT PC10
PC-PT PC11
PC-PT PC12
2960-24TT S3
PC-PT PC13
PC-PT PC14
PC-PT PC5
PC-PT PC6
PC-PT PC7
2960-24TT S2
2911 R2
PC-PT PC8
PC-PT PC9
PC-PT PC15
PC-PT PC16
2960-24TT S4
PC-PT PC17
PC-PT PC18
PC-PT PC19
Realtime
Simulation
Time: 00:30:13
Search for device
Serial DCE
Scenario 0
New
Delete
Toggle PDU List Window
Fire Last Status Source Destination Type Color Time(sec) Periodic Num Edit Delete
31°C
Mostly sunny
Search
ENG
IN

3/10/26 Exp no: 10a

Internetworking with routers in CISCO packet tracer simulator

Aim

To design and configure a simple Internetwork using a routers in CISCO packet tracer.

Procedure :

Step 1:

Configure router1

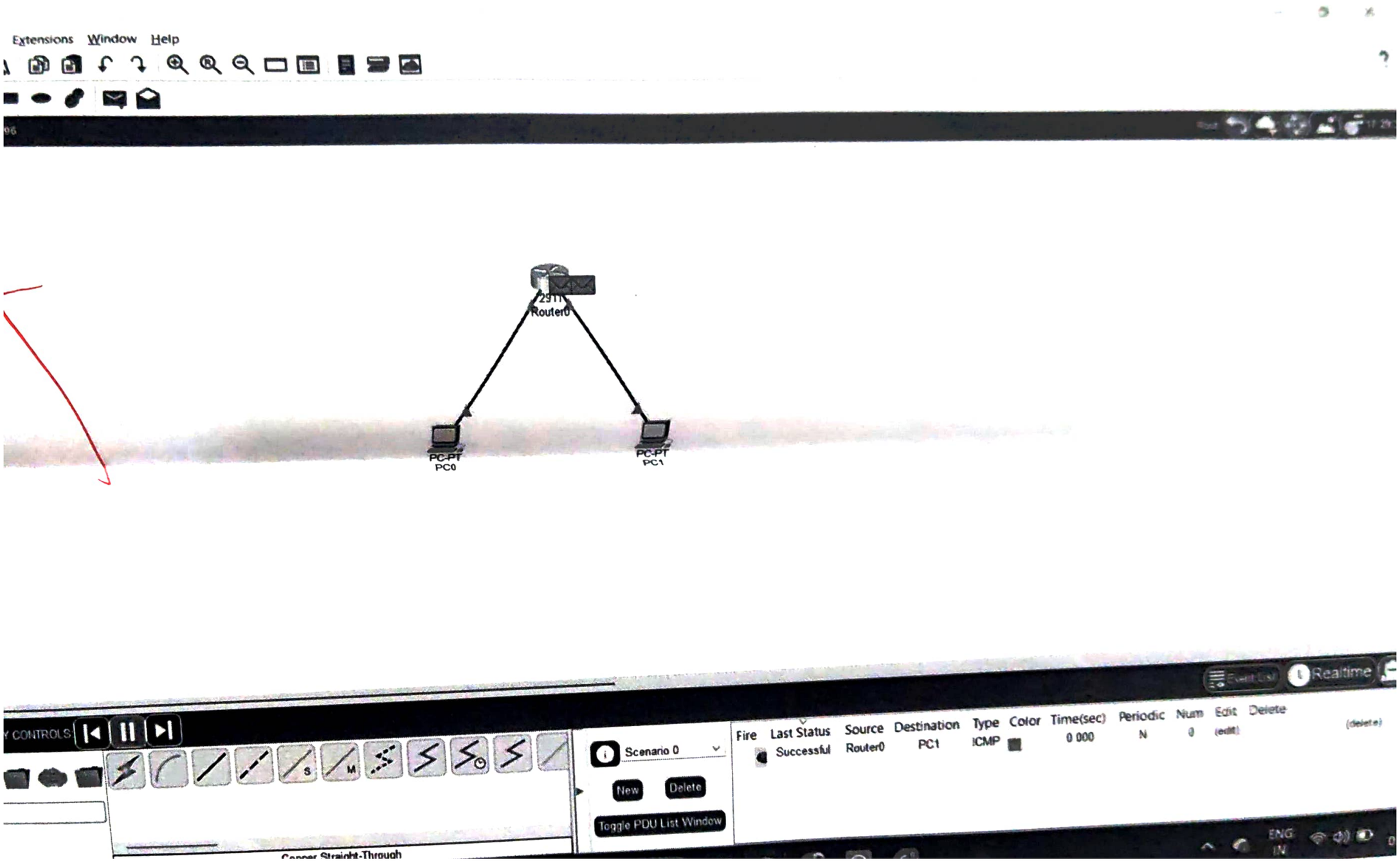
* Select the routers and open CLI
* Press enter to start Configuring Router1
* Type enable to activate the privileged mode.

Step 2:

* Assign IP address to every PC in the network
* Select the PC, go to desktop and select IP Configuration.
* Assign the default gateway of PC's 192.168.10.1
* Assign the default gateway of PC as 192.168.20.1

Step 3:

* Connect fast ethernet 0 port of PC0 with fast ethernet 0/0 port of router1 using a copper straight through cable


Extensions Window Help
2911
Router0
PC-PT
PC0
PC-PT
PC1
Realtime
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete
Successful
Router0
PC1
ICMP
0.000
N
0
(edit)
(delete)
Copper Straight-Through

wireless router, DHCP server, network
Internet cloud.

design and configure an Internetwork
using a wireless routers, DHCP server,
and Internet cloud on CISCO packet
tracer.

cedure
open cisco packet tracer and create
the required network topology.
connect PC, laptop, wireless router,
cable.
configure the wireless router
with SSID homenetwork.
Connect the laptop to the wireless
network and PC to DHCP
verify connectivity using ip config
and ping.

Result:

The Internetwork was
successfully designed and
configured

11a :

26 Simulate static routing configuration using CISCO packet tracer

Aim

To simulate a static routing configuration using Cisco packet tracer.

Procedure

Open Cisco packet tracer and create the given network topology.

* Connect 3 routers and PC's using suitable cable.
* Configure IP address for routers and PC's
* Enable all router interface using no shutdown
* configure static routes on Router 0, router 1, router 2 using IP mobile.
* Test connectivity
* use trace to verify the routing path.
* Disconnect the main link and verify that the backup router works
* Thus a static routing is successfully configured and verified.

Result:

Thus static routing was executed.

3| 11b: 10/2b Simulate RIP using Cisco packet Tracer.

Aim

To simulate RIP using Cisco packet tracer.

Procedure

* Create the topology with 3 routers and 2PCs
* connect the device using suitable cables
* Assign the given IP address to both PCs
* Configure fast ethernet and serial interface of all routers
* Enable interface using no shutdown
* Set clock rate of 64000 and bandwidth on DE serial interface.
* Enable RIP on all routers using router RIP
* Add directly connected network using the network command.
* Test connectivity using ping 20.0.0.2
* use tracer to 20.0.0.2 to verify the selected route.
* Disconnect the direct route 0-router 2 link and use tracer again to verify RIP results the alternate route.

Exp: 12 End-End Communication at transport layer
8/10/26

Aim

To implement TCP and UDP echo client server communication and a chat program using socket programming.

tcp-echo.py

```
def start_server():
    server = socket.socket(socket.AF_INET,
                           socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen(1)
    print(f"[SERVER] Connected by {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() ==
                                'quit':
            print("[SERVER] Connection
                        closed by client.")
            break
        print(f"[SERVER] Received: {data}")
        conn.send(f"Server received:
                    {data}".
                    encode())
conn.close()
server.close()
```

```
def start_client():
    client = socket.socket(socket.AF_INET,
                           socket.SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] connected to server")

    while True:
        msg = input("Enter message (or 'quit'
                    to exit): ")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
        print(f"[SERVER RESPONSE] {response}
                                        ")
    client.close()

if __name__ == "__main__":
    server_thread = threading.Thread
        (target = start_server,
         daemon = True)
    server_thread.start()
    start.client()
```

output:

```
======= Restart: C:/Users/Tkala/
        Downloads/tcp.py =======
[SERVER] listening on 127.0.0.1:5000
                [CLIENT] Connected to server
[SERVER] connected by ('127.0.0.1', 53458)
                Enter message (or 'quit' to
                                exit):
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
```

```
        print("[CLIENT] closing connection...")
        break

    data, _ = c.recvfrom(1024)
    print("Echo from server:", data.decode())

c.close()
print("[CLIENT] stopped.")

if __name__ == "__main__":
    threading.Thread(target=udp_server, daemon=True).start()
    udp_client()
```

output:

```
Enter message ('quit' to stop): [SERVER] Running on 127.0.0.1:12000
hi
[SERVER] Received from ('127.0.0.1', 49670): hi
Enter message ('quit' to stop): quit
[CLIENT] closing connection... [SERVER] client ('127.0.0.1', 49670) ended the connection.
[CLIENT] stopped. [SERVER] stopped.
```

Result:

TCP and UDP echo client server programs executed successfully.

Result:

The ping program was executed successfully.

Exp: 14
9/10/26

write a code using RAW sockets to implement packet sniffing

Aim:

To capture and analyse network packets and display their protocols, source and dest IP addresses.

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP,
                              UDP, ICMP
def packets_callback(packet):
    if IP in packet:
        IP_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dest_ip = ip_layer.dest

        protocol_name = ""
        if protocol == 1:
            protocol_name = "ICMP"
        elif protocol == 6:
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "unknown Protocol"
        print(f"Protocol: {protocol_name}")
        print(f"Source IP: {src_ip}")
        print(f"Destination IP: {dest_ip}")
        print("-" * 50)
```